Build a plugin

Plugins let you extend Obsidian with your own features to create a custom note-taking experience.

In this tutorial, you'll compile a sample plugin from source code and load it into Obsidian.

What you'll learn

After you've completed this tutorial, you'll be able to:

- Configure an environment for developing Obsidian plugins.
- Compile a plugin from source code.
- Reload a plugin after making changes to it.

Prerequisites

To complete this tutorial, you'll need:

- Git installed on your local machine.
- A local development environment for Node.js.
- A code editor, such as Visual Studio Code.

Before you start

When developing plugins, one mistake can lead to unintended changes to your vault. To prevent data loss, you should never develop plugins in your main vault. Always use a separate vault dedicated to plugin development.

Create an empty vault.

Step 1: Download the sample plugin

In this step, you'll download a sample plugin to the `plugins` directory in your vault's `.obsidian` directory so that Obsidian can find it.

The sample plugin you'll use in this tutorial is available in a GitHub repository.

1. Open a terminal window and change the project directory to the `plugins` directory.

```
cd path/to/vault
mkdir .obsidian/plugins
cd .obsidian/plugins
```

2. Clone the sample plugin using Git.

```
git clone https://github.com/obsidianmd/obsidian-sample-plugin.git
```

GitHub template repository

The repository for the sample plugin is a GitHub template repository, which means you can create your own repository from the sample plugin. To learn how, refer to Creating a repository from a template.

Remember to use the URL of your own repository when cloning the sample plugin.

Step 2: Build the plugin

In this step, you'll compile the sample plugin so that Obsidian can load it.

1. Navigate to the plugin directory.

```
cd obsidian-sample-plugin
```

2. Install dependencies.

```
npm install
```

3. Compile the source code. The following command keeps running in the terminal and rebuilds the plugin when you modify the source code.

```
npm run dev
```

Notice that the plugin directory now has a `main.js` file that contains a compiled version of the plugin.

Step 3: Enable the plugin

To load a plugin in Obsidian, you first need to enable it.

1. In Obsidian, open **Settings**.
2. In the side menu, select **Community plugins**.
3. Select **Turn on community plugins**.
4. Under **Installed plugins**, enable the **Sample Plugin** by selecting the toggle button next to it.

You're now ready to use the plugin in Obsidian. Next, we'll make some changes to the plugin.

Step 4: Update the plugin manifest

In this step, you'll rename the plugin by updating the plugin manifest, `manifest.json`. The manifest contains information about your plugin, such as its name and description.

1. Open `manifest.json` in your code editor.
2. Change `id` to a unique identifier, such as `"hello-world"`.
3. Change `name` to a human-friendly name, such as `"Hello world"`.
4. Rename the plugin folder to match the plugin's `id`.
5. Restart Obsidian to load the new changes to the plugin manifest.

Go back to **Installed plugins** and notice that the name of the plugin has been updated to reflect the changes you made.

Remember to restart Obsidian whenever you make changes to `manifest.json`.

Step 5: Update the source code

To let the user interact with your plugin, add a *ribbon icon* that greets the user when they select it.

1. Open `main.ts` in your code editor.
2. Rename the plugin class from `MyPlugin` to `HelloWorldPlugin`.
3. Import `Notice` from the `obsidian` package (if it hasn't been imported already).

```
import { Notice, Plugin } from 'obsidian';
```

4. In the `onload()` method, add the following code:

```
this.addRibbonIcon('dice', 'Greet', () => {
  new Notice('Hello, world!');
```

```
});
```

5. In the **Command palette**, select **Reload app without saving** to reload the plugin.

You can now see a dice icon in the ribbon on the left side of the Obsidian window. Select it to display a message in the upper-right corner.

Remember, you need to **reload your plugin after changing the source code**, either by disabling it then enabling it again in the community plugins panel, or using the command palette as detailed in part 5 of this step.

> **Hot reloading**
>
> Install the Hot-Reload plugin to automatically reload your plugin while developing.

Conclusion

In this tutorial, you've built your first Obsidian plugin using the TypeScript API. You've modified the plugin and reloaded it to reflect the changes inside Obsidian.

LINKS TO THIS PAGE

Build a Bases view
Home
Use Svelte in your plugin